

KONEČNÉ AUTOMATY

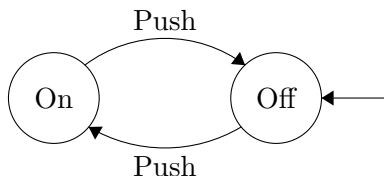
V této kapitole se budeme věnovat základům z tzv. *teorie automatů*. Konečné automaty jsou základním konceptem, který nachází široké uplatnění jak v informatice, tak v matematice. Tyto jednoduché abstraktní stroje jsou schopné rozpoznávat a generovat tzv. *formální jazyky*, a tím poskytují základní model pro analýzu a syntézu tzv. *regulárních jazyků*. V této kapitole se seznámíme s různými typy konečných automatů, jejich vlastnostmi a později i praktickými aplikacemi (viz kapitola o kompilátorech).

Důvodem pro nás bude, abychom lépe porozuměli pojmu “výpočet”. Čtenář možná již někdy slyšel o vývojovém diagramu v případě návrhu nějakého programu. Konečné automaty jsou v tomto ohledu alternativní nástrojem pro jeho popis, avšak jejich princip spočívá ve zpracovávání vstupu jako posloupnosti znaků. V úvodu si tedy představíme několik nových termínů souvisejících s formálními jazyky a posléze si představíme základní dvojici konečných automatů.

Jako zajímavost zde uvedme, že teorie automatů též hodně zasahuje do teorie složitosti, kterou jsme v základu pokryli v kapitole o algoritmicky těžkých problémech. Díky automatům je tedy možný i alternativní přístup k dané problematice. Tím se zde již však zabývat nebudeme.

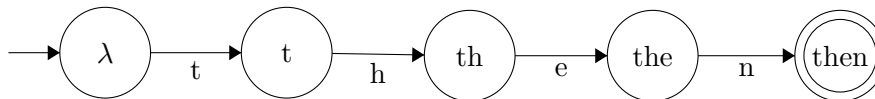
1 Úvod

Konečné (stavové) automaty představují abstraktní stroje reprezentující určitý výpočet nebo algoritmus. Z reálného světa můžeme za stavový automat považovat např. obyčejný vypínač, viz obrázek 1. Příkladem



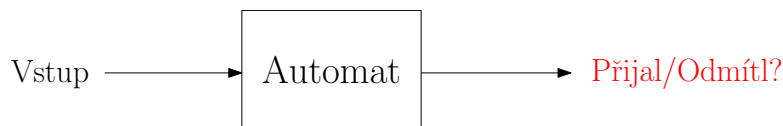
Obrázek 1: Schéma automatu pro vypínač.

praktického použití návrhu automatů je tzv. *lexikální analýza*, kterou se budeme zabývat v kapitole o kompilátorech. Na obrázku 2 je vidět příklad konečného automatu rozpoznávající slovo *then*.



Obrázek 2: Schéma automatu rozpoznávající slovo **then**.

Jak tedy vlastně konečné automaty fungují? Vstup je reprezentován určitou **konečnou posloupností znaků**, tedy nějakým řetězcem neboli tzv. **slovem**, které daný automat zpracovává postupně po jednotlivých znacích. Na konci výpočtu vydá automat odpověď, zda vstup **přijal**, či **nepřijal** (viz obrázek 3).



Obrázek 3: Zjednodušené schéma konečného automatu.

2 Formální jazyky

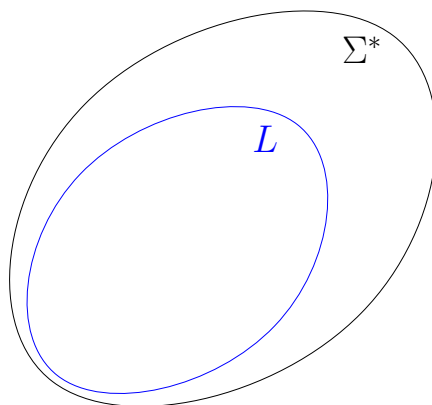
Nyní, když jsme si vysvětlili základy konečných automatů a jejich roli ve zpracovávání formálních jazyků, můžeme se hlouběji ponořit do problematiky formálních jazyků samotných. Tyto jazyky jsou totiž základem pro popis množin řetězců a jsou klíčové pro definování toho, co automaty rozpoznávají.

Formální jazyky lze chápat jako abstraktní systémy pro popis pravidel, podle kterých jsou vytvářeny řetězce symbolů. Jsou zásadní nejen pro teorii automatů, ale také pro návrh programovacích jazyků a kompilátorů. V následujících částech se zaměříme na různé typy gramatik, které jsou základem pro definici formálních jazyků, a ukážeme si, jak konečné automaty mohou tyto jazyky rozpoznávat a zpracovávat.

Pro začátek si (jako vždy) uvedeme definice některých základních pojmů.

Definice 2.1. Máme neprázdnou množinu znaků Σ , tzv. *abecedu*.

- *Slovo (řetězec)* je libovolná konečná posloupnost znaků ze Σ .
- *Prázdné slovo* je slovo neobsahující žádné znaky¹, značíme λ .
- *Množina všech slov v abecedě Σ* se značí Σ^* .
- *Jazyk* je libovolná podmnožina slov $L \subseteq \Sigma^*$. Říkáme, že jazyk L je *nad abecedou Σ* (viz obrázek 4).



Obrázek 4: Obecný jazyk L nad abecedou Σ .

- *Délka slova w* (tzn. počet znaků) se značí $|w|$.
- *Konkatenace slov $u, v \in \Sigma^*$* , kde $u = u_1u_2 \dots u_n$ a $v = v_1v_2 \dots v_m$ je slovo

$$uv = u_1u_2 \dots u_nv_1v_2 \dots v_m.$$

- *n -tá mocnina slova $u \in \Sigma^*$* je slovo

$$u^n = \underbrace{uu \dots u}_{n\text{-krát}},$$

¹Symbol λ zde představuje speciální znak zastupující prázdné slovo. Neuvažujeme jej tedy jako součást abecedy Σ , tzn. $\lambda \notin \Sigma$.

kde $n \in \mathbb{N}_0$. Speciálně $u^0 = \lambda$.

Podívejme se na jednoduchý příklad.

Příklad 2.2. Mějme dvouznačkovou abecedu $\Sigma = \{a, b\}$.

- Např. slovem w takto zadané abecedě Σ je $w = abba$. Tzn. $|w| = 4$.
- Množina všech slov² $\Sigma^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, aab, aba, \dots\}$.
- Jednoduchý příklad jazyka nad abecedou Σ je $L_1 = \Sigma^*$, tedy všechna možná slova sestávající ze znaků a, b .
- Dalším příkladem může být např. jazyk $L_2 = \{w \in \Sigma^* \mid |w| \leq 2\}$. Jazyk L_2 tedy obsahuje všechna slova délky nejvýše 2, tj.

$$L_2 = \{\lambda, a, b, aa, ab, ba, bb\}.$$

- Pátá mocnina slova $w = abba$:

$$w^5 = abbaabbaabbaabbaabba.$$

- Konkatenace slov $u = aabb$ a $v = bbaa$

$$uv = aabbbbbaa.$$

Jazyky nejsou v matematickém pojetí nic jiné než množiny objektů, tedy v tomto případě slov.

Příklad 2.3. Další příklady jazyků:

- Jazyk $U = \{w \in \Sigma^* \mid w \subseteq \{X, Y\}^*\}$ obsahující všechna slova sestávající pouze z velkých písmen nad abecedou $\Sigma = \{x, y, X, Y\}$.
- Jazyk $N = \{u \in \Lambda^* \mid |u| \geq 1\}$ obsahující všechna neprázdná slova nad obecnou abecedou Λ (může být jakákoliv).
- Jazyk $Z = \{0^n 1^n \mid n \in \mathbb{N}\}$ nad abecedou $\Omega = \{0, 1\}$.

3 Deterministické konečné automaty

Již jsme si představili jazyky jakožto prostředek pro reprezentaci vstupů pro konečné automaty. Stále jsme si ovšem neřekli, co to ten automat vlastně je. Podobně jako v definici grafu, kdy máme uspořádanou dvojici vrcholů a hran, i zde bude myšlenka velice podobná.

Definice 3.1 (Deterministický končný automat). *Deterministický konečný automat* je uspořádaná pětice $A = (Q, \Sigma, \delta, q_0, F)$, kde

- Q je konečná množina stavů,
- Σ je abeceda, z níž sestávají slova na vstupu,
- δ je tzv. *přechodová funkce* $Q \times \Sigma \rightarrow Q$ udávající přechody mezi stavy,
- $q_0 \in Q$ je *počáteční stav* automatu
- a $F \subseteq Q$ je *množina přijímajících stavů*.

Automaty zakreslujeme pomocí stavového diagramu (formálně orientovaného grafu), přičemž

²Je pochopitelné, že tato množina je nekonečná (délka slov není nijak omezená).

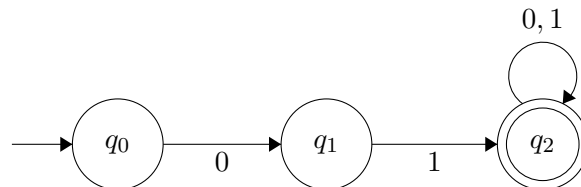
- *vrcholy* jsou jednotlivé stavy automatu (dvojitý kruh značí přijímající stav a do počátečního stavu vede šipka “odnikud”)
- a *šipky* reprezentují přechody mezi stavy.

Na začátku výpočtu dostane automat na vstupu slovo $w \in \Sigma^*$, které čte znak po znaku (zleva doprava). Začínáme v počátečním stavu q_0 , přičemž vždy podle následujícího čteného znaku se automat přesouvá do nového stavu. Přechodová funkce $\delta : Q \times \Sigma \rightarrow Q$ nám udává pro každou dvojici (q, a) , kde q je stav, v němž se automat nachází a a je aktuálně čtený znak, do jakého stavu se má automat přesunout. Pokud na konci čtení slova w skončí výpočet v jednom z přijímajících stavů, pak automat slovo přijímá. V opačném případě, kdy výpočet neskončí v přijímajícím stavu nebo přechod pro určitý znak z určitého stavu není definovaný, pak výpočet končí a automat slovo nepřijímá.

Příklad 3.2. Máme automat $A = (Q, \Sigma, \delta, q_0, F)$, kde

- $Q = \{q_0, q_1, q_2\}$,
- q_0 je počáteční stav,
- $F = \{q_2\}$,
- $\Sigma = \{0, 1\}$,
- Přechodová funkce δ (viz tabulka 1):
 - $\delta(q_0, 0) = q_1$,
 - $\delta(q_0, 1) = q_2$,
 - $\delta(q_1, 0) = q_2$,
 - $\delta(q_1, 1) = q_2$.

Schéma automatu lze vidět na obrázku 5 níže.



Obrázek 5: Schéma automatu A a tabulka přechodové funkce δ

	0	1
q_0	q_1	–
q_1	–	q_2
q_2	q_2	q_2

Tabulka 1: Tabulka přechodové funkce δ automatu A

- 4 **Nedeterministické konečné automaty**
- 5 **Regulární výrazy**
- 6 **Gramatiky**